

Q53. Design Patterns Used in Automation

Senior SDET / Test Automation Lead — Interview-Ready Answer

1. Interview-ready answer

“In automation frameworks, I use design patterns to make the framework maintainable, reusable and scalable. The most relevant patterns are Page Object Model for UI abstraction, Factory for creating different browser or client implementations, Builder for constructing complex test data, Strategy for interchangeable behaviours, and dependency injection or fixtures for lifecycle management. I use Singleton carefully, mainly for safe immutable configuration, rather than for WebDriver because shared WebDriver creates problems with parallel execution.”

2. Page Object Model (POM)

Purpose: Separate test/business behaviour from UI implementation details.

```
public class LoginPage { private WebDriver driver; private By username =  
By.id("username"); private By password = By.id("password"); private By loginButton =  
By.id("login"); public LoginPage(WebDriver driver) { this.driver = driver; } public void  
login(String user, String pwd) { driver.findElement(username).sendKeys(user);  
driver.findElement(password).sendKeys(pwd); driver.findElement(loginButton).click(); } }
```

Interview point: If a locator changes, I update the Page Object rather than modifying every test.

3. Factory Pattern

Purpose: Centralise object creation, such as Chrome, Firefox or Edge drivers.

```
public class DriverFactory { public static WebDriver createDriver(String browser) { if  
(browser.equalsIgnoreCase("chrome")) return new ChromeDriver(); if  
(browser.equalsIgnoreCase("firefox")) return new FirefoxDriver(); if  
(browser.equalsIgnoreCase("edge")) return new EdgeDriver(); throw new  
IllegalArgumentException("Unsupported browser: " + browser); } }
```

Interview point: The caller does not need to know how the WebDriver is created.

4. Builder Pattern

Purpose: Build complex test data or API request objects with many optional fields.

```
User user = new User.Builder() .firstName("Swapnil") .lastName("Bodade")  
.email("test@test.com") .state("GA") .build();
```

Interview point: Builder improves readability and avoids constructors with many parameters.

5. Strategy Pattern

Purpose: Define multiple implementations of the same behaviour and select one at runtime.

```
public interface LoginStrategy { void login(); } public class PasswordLogin implements  
LoginStrategy { public void login() { System.out.println("Password login"); } } public  
class OTPLogin implements LoginStrategy { public void login() { System.out.println("OTP  
login"); } }
```

Automation examples: password/OTP/SSO authentication, different API authentication mechanisms, environment-specific behaviour, or alternative data-generation strategies.

6. Singleton Pattern — use carefully

A shared Singleton WebDriver is risky in parallel execution because multiple tests can interfere with the same browser session. For parallel frameworks, prefer isolated drivers/contexts, such as ThreadLocal in Selenium or isolated Playwright browser contexts/fixtures.

Good use: safe immutable configuration or another genuinely application-wide resource.

7. Dependency Injection / Fixtures

Purpose: Keep object creation and lifecycle management outside Page Objects.

```
public LoginPage(WebDriver driver) { this.driver = driver; }
```

The framework or fixture creates the browser and injects it into the Page Object. This improves loose coupling and makes lifecycle management easier.

8. How I would relate these patterns to my Selenium → Playwright migration

“During the Selenium-to-Playwright migration, I did not simply convert Selenium syntax line by line. We used the migration as an opportunity to improve the framework architecture. POM remained the UI abstraction, browser creation and environment-specific setup were centralised, selectors were separated into configuration, and browser/page lifecycle was handled through framework fixtures. We also used isolated browser contexts to support reliable parallel execution.”

9. Important interview distinction

Pattern	Automation problem it solves
POM	Separates UI implementation from test logic
Factory	Centralises browser/client object creation
Builder	Creates complex test data/readable objects
Strategy	Supports interchangeable behaviours
Singleton	One shared instance — use cautiously
DI / Fixtures	Manages dependencies and lifecycle

10. Strong final answer to memorise

“In my automation frameworks, POM is the primary pattern for UI abstraction. I use Factory-style design for browser or client creation, Builder for complex test data, and Strategy where the same operation has multiple implementations. I use dependency injection or fixtures for lifecycle management. I am cautious with Singleton: I would not use a global Singleton WebDriver in a parallel framework because it creates shared mutable state. During my Selenium-to-Playwright migration, these patterns helped us make the framework more modular, maintainable and suitable for parallel execution.”

11. Likely follow-up questions

- What is the difference between Factory and Abstract Factory?
- Why should WebDriver not be a Singleton in parallel execution?
- Is Page Object Model a design pattern?
- Factory vs Strategy — what is the difference?
- Where have you used Builder in your framework?
- How does dependency injection work in Playwright?
- Which design patterns did you use during your Selenium → Playwright migration?
- What design patterns are used internally by Selenium or Playwright?
- What is the difference between a design pattern and an architectural pattern?